

Animales

Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén Sánchez, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

Índice

1. Objetivos	2
1.1. Obj-1: Objetivo 1	2
1.2. Obj-2: Objetivo 2	2
2. Requisitos de información (RI)	3
2.1. RI-1: Animales	3
2.2. RI-2: Adopciones	3
3. Reglas de negocio (RN)	4
3.1. RN-1: Límite de adopciones	4
3.2. RN-2: Datos obligatorios según tipo de ingreso	4
4. Diagrama de clases	6
5. Álgebra relacional	9
6. Script SQL para crear la base de datos	11
7. Script SQL para la carga inicial de datos	14
8. Consultas	17
9. Triggers	19
10. Pruebas SQL	21

Requisitos

Objetivos

1.1. Obj-1: Objetivo 1

- Responsable: Responsable de CAA
- Descripción: Llevar a cabo el ingreso de un animal, abandonado en la vía pública o entregado por una persona, en cuyo caso habría que identificarla, clasificándolo por razas y especies
- Para: Mantener los animales custodiados y ofrecerlos en adopción a personas interesadas

1.2. Obj-2: Objetivo 2

- Responsable: Responsable de CAA
- Descripción: Gestionar consultas de animales disponibles y adjudicar adopciones
- Para: Adjudicar cuanto antes los animales existentes y disminuir costes de mantenimiento del CAD

Requisitos de información (RI)

2.1. RI-1: Animales

- Como: Operario de Recogida de Animales
- Quiero: Conocer el nombre de cada especie y nombre de cada raza dentro de una especie. Un animal ingresado puede tener un nombre y, opcionalmente, un microchip. Siempre se especificará la fecha y hora del ingreso y una descripción adicional del animal. Si lo ha entregado una persona se la identificará (especificando nombre, dirección y email); en caso contrario se especificará como abandono del animal en la vía pública, describiendo, obligatoriamente, el lugar y hora del hallazgo
- Para: Ingresar cada mascota con la clasificación adecuada y/o conocer datos sobre el hallazgo

2.2. RI-2: Adopciones

- Como: Operario de Adopciones
- Quiero: Registrar adopciones especificando persona que realiza la adopción de un animal disponible y fecha y hora en que se produce
- Para: Dejar constancia de las adopciones realizadas por el CAD

Reglas de negocio (RN)

3.1. RN-1: Límite de adopciones

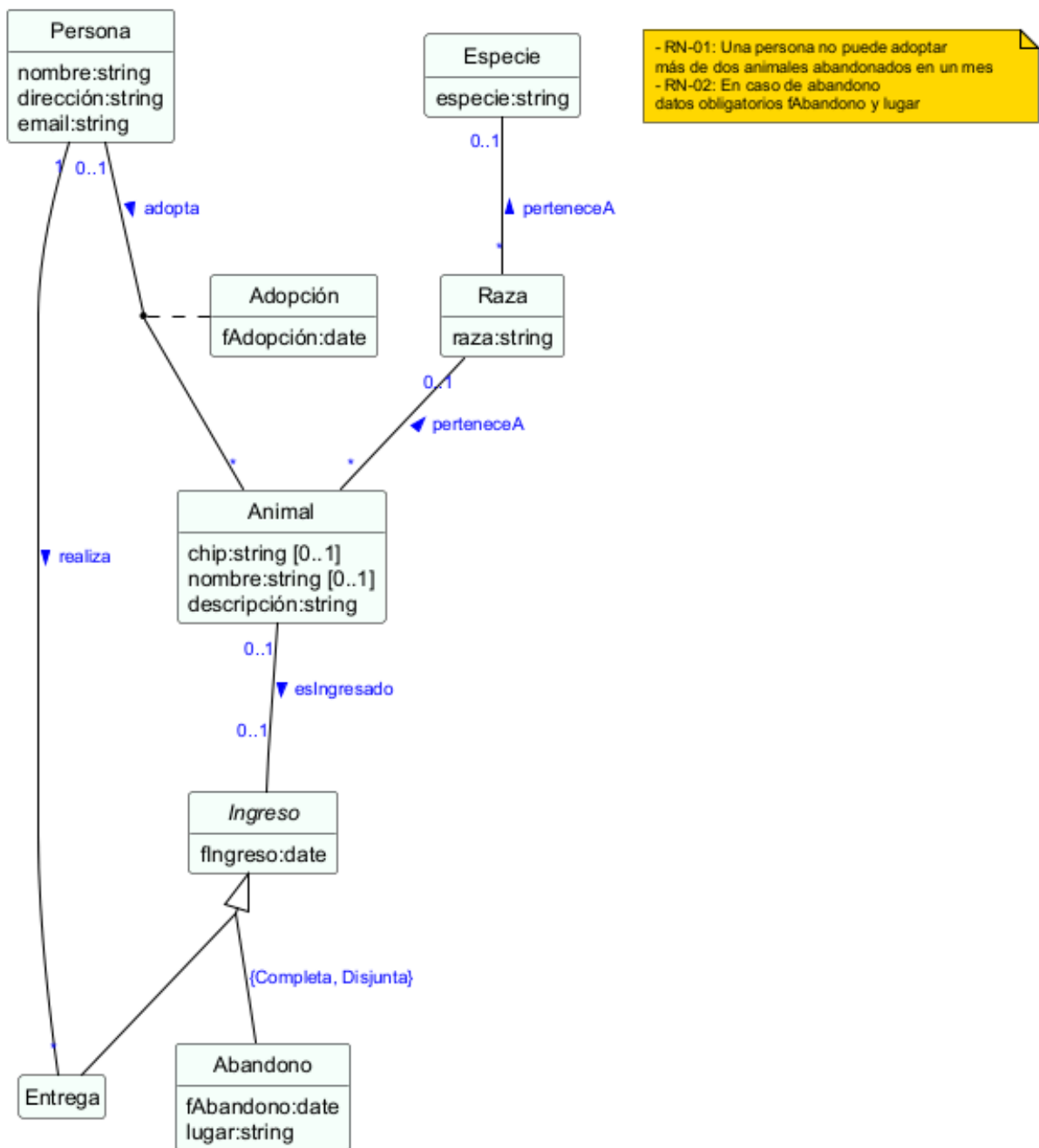
- Como: Responsable del CAA
- Quiero: Que una persona no pueda adoptar más de dos animales abandonados en un mes
- Para: Poder evitar que se concentren adopciones en personas

3.2. RN-2: Datos obligatorios según tipo de ingreso

- Como: Responsable de la Oficina
- Quiero: Si un animal es hallado “abandonado” en la vía pública, entonces es obligatorio especificar el “lugar, fecha y hora de encuentro”, en caso contrario se trata de una “entrega” y hay que identificar la persona que la realiza (nombre, dirección y email)
- Para: Facilitar las búsquedas de sus dueños

Modelo Conceptual

Diagrama de clases



4. Diagrama de clases

```
Especies = { especieId, especie }
  PK(especieId)
  AK(especie)
Razas = { razaId, especieId, raza }
  PK(razaId)
  FK(especieId) / Especies
  AK(especieId, raza)
Animales = { animalId, razaId, chip, nombre, descripcion }
  PK(animalId)
  FK(razaId) / Razas
  AK(chip)
Personas = { personaId, nombre, direccion, email }
  PK(personaId)
  AK(email)
Ingresos = { ingresoId, personaId, animalId, fIngreso }
  PK(ingresoId)
  FK(personaId) / Personas
  FK(animalId) / Animales
  AK(ingresoId, animalId)
Entregas = { ingresoId }
  PK(ingresoId)
  FK(ingresoId) / Ingresos
Abandonos = { ingresoId, fAbandono, lugar }
  PK(ingresoId)
  FK(ingresoId) / Ingresos
Adopciones = { adopcionId, personaId, animalId, fAdopción }
  PK(adopcionId)
  FK(personaId) / Personas
  FK(animalId) / Animales
  -- Una persona no puede adoptar más de dos animales abandonados en un mes.

Especies = {
  (e1, "Canino"),
  (e2, "Felino")
}

Razas = {
  (r1, e1, "Labrador"),
  (r2, e1, "Pastor Alemán"),
  (r3, e2, "Siamesa")
}

Animales = {
  (a1, r1, "12345", "Max", "Perro marrón"),
  (a2, r1, "67890", "Rocky", "Perro negro"),
  (a3, r3, "54321", "Luna", "Gata gris")
}

Personas = {
  (p1, "Ana García", "Calle Sol 1", "ana@example.com"),
  (p2, "Luis Pérez", "Calle Luna 2", "luis@example.com"),
  (p3, "Marta López", "Calle Mar 3", "marta@example.com")
}

Ingresos = {
  (i1, p1, a1, "2024-09-15 10:00"),
  (i2, p3, a2, "2024-10-01 09:30"),
  (i3, p2, a3, "2024-10-05 18:15")
}

Entregas = {
  (i1),
  (i3)
```

```
}  
  
Abandonos = {  
  (i2, "2024-10-01 08:45", "Parque de María Luisa")  
}  
  
Adopciones = {  
  (ad1, p2, a1, "2024-10-20 12:00"),  
  (ad2, p3, a2, "2024-10-22 17:30"),  
  (ad3, p1, a3, "2024-11-03 11:00")  
}
```

Álgebra relacional

- Renombrado:

$$P \leftarrow \rho_{P(pid,pn,dir,email)} (Personas)$$

$$A \leftarrow \rho_{A(aid,rid,chip,an,desc)} (Animales)$$

$$Ad \leftarrow \rho_{Ad(adid,pid,aid,fha)} (Adopciones)$$

$$I \leftarrow \rho_{I(iid,pid,aid,fhe)} (Ingresos)$$

- Personas que adoptan animales:

$$PerAdoAni \leftarrow P \bowtie Ad \bowtie A$$

```
PerAdoAni = { pid, pn, dir, email, adid, aid, fha, rid, chip, an, desc }
```

```
PerAdoAni = {
  (p2, "Luis Pérez", "Calle Luna 2", "luis@example.com", ad1, a1, "2024-10-20 12:00", r1,
  "12345", "Max", "Perro marrón"),
  (p3, "Marta López", "Calle Mar 3", "marta@example.com", ad2, a2, "2024-10-22 17:30", r1,
  "67890", "Rocky", "Perro negro"),
  (p1, "Ana García", "Calle Sol 1", "ana@example.com", ad3, a3, "2024-11-03 11:00", r3,
  "54321", "Luna", "Gata gris")
}
```

- Personas que entregan animales:

$$PerEntAni \leftarrow P \bowtie I \bowtie A$$

```
PerEntAni = { pid, pn, dir, email, iid, aid, fhe, rid, chip, an, desc }
```

```
PerEntAni = {
  (p1, "Ana García", "Calle Sol 1", "ana@example.com", i1, a1, "2024-09-15 10:00", r1,
  "12345", "Max", "Perro marrón"),
  (p3, "Marta López", "Calle Mar 3", "marta@example.com", i2, a2, "2024-10-01 09:30", r1,
  "67890", "Rocky", "Perro negro"),
  (p2, "Luis Pérez", "Calle Luna 2", "luis@example.com", i3, a3, "2024-10-05 18:15", r3,
  "54321", "Luna", "Gata gris")
}
```

- Personas que entregan animales abandonados:

$$PerEntAniAba \leftarrow P \bowtie I \bowtie_{Ab(iid, fhab, lugar)}^{\rho} (Abandonos) \bowtie A$$

```
PerEntAniAba = { pid, pn, dir, email, iid, aid, fhe, fhab, lugar, rid, chip, an, desc }
```

```
PerEntAniAba = {
  (p3, "Marta López", "Calle Mar 3", "marta@example.com", i2, a2, "2024-10-01 09:30",
  "2024-10-01 08:45", "Parque de María Luisa", r1, "67890", "Rocky", "Perro negro")
}
```

- Adopciones en octubre:

$$Mes(fha) = \text{Octubre} \quad \sigma_{Mes(fha) = \text{Octubre}} (PerAdoAni)$$

```
AdopcionesOctubre = { pid, pn, dir, email, adid, aid, fha, rid, chip, an, desc }
```

```
AdopcionesOctubre = {
  (p2, "Luis Pérez", "Calle Luna 2", "luis@example.com", ad1, a1, "2024-10-20 12:00", r1,
  "12345", "Max", "Perro marrón"),
  (p3, "Marta López", "Calle Mar 3", "marta@example.com", ad2, a2, "2024-10-22 17:30", r1,
  "67890", "Rocky", "Perro negro")
}
```

- Adopciones por especie:

$$\gamma_{especieId, especie, \rho_{total}(\text{COUNT}(*))} (especieId, especie(PerAdoAni \bowtie Razas \bowtie Especies))$$

```
AdopcionesPorEspecie = { especieId, especie, total }
```

```
AdopcionesPorEspecie = {
  (e1, "Canino", 2),
  (e2, "Felino", 1)
}
```

Modelo Tecnológico

Script SQL para crear la base de datos

animales/createDB.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Octubre de 2024  
-- Descripción: Script para crear la BD del ejercicio de Animales  
--  
  
DROP DATABASE IF EXISTS AnimalesDB;  
CREATE DATABASE AnimalesDB;  
USE AnimalesDB;  
  
-- Tabla species  
CREATE OR REPLACE TABLE species (  
    species_id INT AUTO_INCREMENT,  
    species_name VARCHAR(255) NOT NULL,  
    PRIMARY KEY (species_id),  
    UNIQUE KEY (species_name)  
);  
  
-- Tabla razas  
CREATE OR REPLACE TABLE breeds (  
    breed_id INT AUTO_INCREMENT,  
    species_id INT,  
    breed_name VARCHAR(255) NOT NULL,
```

```

PRIMARY KEY (breed_id),
FOREIGN KEY (species_id) REFERENCES species(species_id),
UNIQUE KEY (species_id, breed_name)
);

-- Tabla Animales
CREATE OR REPLACE TABLE animals (
    animal_id INT AUTO_INCREMENT,
    breed_id INT,
    chip VARCHAR(255),
    name VARCHAR(255),
    description TEXT,
    PRIMARY KEY (animal_id),
    FOREIGN KEY (breed_id) REFERENCES breeds(breed_id),
    UNIQUE KEY (chip)
);

-- Tabla personas
CREATE OR REPLACE TABLE people (
    person_id INT AUTO_INCREMENT,
    name VARCHAR(255),
    address VARCHAR(255),
    email VARCHAR(255) NOT NULL,
    PRIMARY KEY (person_id),
    UNIQUE KEY (email)
);

-- Tabla ingresos
CREATE OR REPLACE TABLE admissions (
    admission_id INT AUTO_INCREMENT,
    person_id INT,
    animal_id INT,
    delivery_datetime DATETIME NOT NULL,
    PRIMARY KEY (admission_id),
    FOREIGN KEY (person_id) REFERENCES people(person_id),
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id),
    UNIQUE KEY (animal_id)
);

-- Tabla entregas
CREATE OR REPLACE TABLE deliveries (
    admission_id INT,
    PRIMARY KEY (admission_id),
    FOREIGN KEY (admission_id) REFERENCES admissions(admission_id)
);

-- Tabla abandonos
CREATE OR REPLACE TABLE abandonments (
    admission_id INT,
    abandonment_datetime DATETIME NOT NULL, -- RN-02

```

```

    place VARCHAR(255) NOT NULL,          -- RN-02
    PRIMARY KEY (admission_id),
    FOREIGN KEY (admission_id) REFERENCES admissions(admission_id)
);

-- Tabla adopciones
CREATE OR REPLACE TABLE adoptions (
    adoption_id INT AUTO_INCREMENT,
    person_id INT,
    animal_id INT,
    adoption_datetime DATETIME,
    PRIMARY KEY (adoption_id),
    FOREIGN KEY (person_id) REFERENCES people(person_id),
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id),
    CONSTRAINT uq_adoptions_animal UNIQUE (animal_id)
    -- UNIQUE KEY (person_id, MONTH(adoption_datetime)) Esto no se puede
    hacer
    -- cuando se crea la tabla, hay que hacerlo con triggers y procedures
);

```

Script SQL para la carga inicial de datos

animales/populateDB.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Septiembre 2026  
-- Descripción: Procedimiento para poblar la BD de Animales  
--  
  
USE AnimalesDB;  
  
DELIMITER //  
CREATE OR REPLACE PROCEDURE p_populate()  
BEGIN  
    SET FOREIGN_KEY_CHECKS = 0;  
    DELETE FROM adoptions;  
    DELETE FROM abandonments;  
    DELETE FROM deliveries;  
    DELETE FROM admissions;  
    DELETE FROM people;  
    DELETE FROM animals;  
    DELETE FROM breeds;  
    DELETE FROM species;  
    SET FOREIGN_KEY_CHECKS = 1;  
  
    INSERT INTO species (species_id, species_name) VALUES
```

```

    (1, 'Canino'),
    (2, 'Felino');

-- Insertar datos en la tabla razas
INSERT INTO breeds (breed_id, species_id, breed_name) VALUES
    (1, 1, 'Labrador'),
    (2, 1, 'Pastor Alemán'),
    (3, 2, 'Siamesa');

-- Insertar datos en la tabla Animales
INSERT INTO animals (animal_id, breed_id, chip, name, description)
VALUES
    (1, 1, '12345', 'Max', 'Perro color marrón'),
    (2, 1, '67890', 'Rocky', 'Perro negro'),
    (3, 2, '54321', 'Luna', 'Gato gris');

-- Insertar datos en la tabla personas
INSERT INTO people (person_id, name, address, email) VALUES
    (1, 'Carlos García', 'Calle False 123',
'carlosgarcia@example.com'),
    (2, 'Laura Martínez', 'Avenida Siempre Viva 456',
'lauramartinez@example.com');

-- Insertar datos en la tabla ingresos
INSERT INTO admissions (admission_id, person_id, animal_id,
delivery_datetime) VALUES
    (1, 1, 2, '2024-09-30 15:00:00'),
    (2, 1, 1, '2024-07-30 15:00:00');

-- Insertar datos en la tabla entregas
INSERT INTO deliveries (admission_id) VALUES
    (1);

-- Insertar datos en la tabla abandonos
INSERT INTO abandonments (admission_id, abandonment_datetime, place)
VALUES
    (2, '2024-07-30 14:30:00', 'Parque Central');

-- Insertar datos en la tabla adopciones
INSERT INTO adoptions (adoption_id, person_id, animal_id,
adoption_datetime) VALUES
    (1, 2, 3, '2024-10-05 00:00:00'),
    (2, 2, 1, '2024-10-10 00:00:00');

-- Datos adicionales
-- Insertar dos razas de perros y otra de gatos
INSERT INTO breeds (breed_id, species_id, breed_name) VALUES
    (4, 1, 'Bulldog'),
    (5, 1, 'Dálmata'),
    (6, 2, 'Persa');

```

```

-- Insertar 2 bulldogs, 1 dálмата y 1 persa:
INSERT INTO animals (animal_id, breed_id, chip, name, description)
VALUES
    (4, 4, '11111', 'Rex', 'Perro blanco'),
    (5, 4, '22222', 'Bobby', 'Perro marrón'),
    (6, 5, '33333', 'Pongo', 'Perro blanco con manchas negras'),
    (7, 6, '44444', 'Misi', 'Gato blanco');

-- Insertar 2 personas más:
INSERT INTO people (person_id, name, address, email) VALUES
    (3, 'Ana López', 'Calle Falsa 456', 'alopez@mail.com'),
    (4, 'Juan Pérez', 'Avenida Siempre Viva 789', 'jperez@mail.com');

-- Insertar un ingreso para los 5 animales con fechas y horas
distintas:
INSERT INTO admissions (admission_id, person_id, animal_id,
delivery_datetime) VALUES
    (3, 3, 4, '2024-10-15 15:00:00'),
    (4, 3, 5, '2024-10-15 15:30:00'),
    (5, 3, 6, '2024-10-15 16:00:00'),
    (6, 4, 7, '2024-10-15 16:30:00');

INSERT INTO deliveries (admission_id) VALUES
    (4),
    (6);

INSERT INTO abandonments (admission_id, abandonment_datetime, place)
VALUES
    (3, '2024-07-30 14:30:00', 'Parque Norte'),
    (5, '2024-07-30 14:30:00', 'Parque Sur');

INSERT INTO adoptions (adoption_id, person_id, animal_id,
adoption_datetime) VALUES
    (3, 3, 4, '2024-10-05 00:00:00'),
    (4, 3, 5, '2024-10-10 00:00:00'),
    (5, 3, 6, '2024-10-15 00:00:00'),
    (6, 4, 7, '2024-10-20 00:00:00');

END //
DELIMITER ;

CALL p_populate();

```

Consultas

animales/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Consultas SQL correspondientes al álgebra relacional de
-- Animales
--

USE AnimalesDB;

-- PerAdoAni: personas que adoptan animales
CREATE OR REPLACE VIEW v_people_adoptions_animals AS
SELECT p.person_id AS pid, p.name AS pn, p.address AS dir, p.email,
       ad.adoption_id AS adid, ad.animal_id AS aid,
       ad.adoption_datetime AS fha,
       a.breed_id AS rid, a.chip, a.name AS an, a.description AS `desc`
FROM people p
JOIN adoptions ad ON ad.person_id = p.person_id
JOIN animals a ON a.animal_id = ad.animal_id;

SELECT * FROM v_people_adoptions_animals;

-- PerEntAni: personas que entregan animales
CREATE OR REPLACE VIEW v_people_admissions_animals AS
SELECT p.person_id AS pid, p.name AS pn, p.address AS dir, p.email,
       i.admission_id AS iid, i.animal_id AS aid, i.delivery_datetime AS
fhe,
```

```

        a.breed_id AS rid, a.chip, a.name AS an, a.description AS `desc`
FROM people p
JOIN admissions i ON i.person_id = p.person_id
JOIN animals a ON a.animal_id = i.animal_id;

SELECT * FROM v_people_admissions_animals;

-- PerEntAniAba: personas que entregan animales abandonados
SELECT pe.pid, pe.pn, pe.dir, pe.email, pe.iid, pe.aid, pe.fhe,
       ab.abandonment_datetime AS fhab, ab.place,
       pe.rid, pe.chip, pe.an, pe.`desc`
FROM v_people_admissions_animals pe
JOIN abandonments ab ON ab.admission_id = pe.iid;

-- AdopcionesOctubre: adopciones realizadas en octubre
SELECT *
FROM v_people_adoptions_animals
WHERE MONTH(fha) = 10;

-- AdopcionesPorEspecie: Número de adopciones por especie
SELECT e.species_id, e.species_name, COUNT(*) AS total
FROM v_people_adoptions_animals pa
JOIN breeds r ON r.breed_id = pa.rid
JOIN species e ON e.species_id = r.species_id
GROUP BY e.species_id, e.species_name;

```

Triggers

animales/triggers.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Triggers y rutinas auxiliares de AnimalesDB
--
USE AnimalesDB;

DELIMITER //
CREATE OR REPLACE TRIGGER t_bi_deliveries_rn02
BEFORE INSERT ON deliveries
FOR EACH ROW
BEGIN
    DECLARE v_person_id INT;

    SELECT person_id INTO v_person_id
    FROM admissions
    WHERE admission_id = NEW.admission_id;

    IF v_person_id IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN02: Una entrega debe identificar a la
persona que la realiza';
    END IF;
END //

CREATE OR REPLACE TRIGGER t_bi_adoptions_rn01
```

```

BEFORE INSERT ON adoptions
FOR EACH ROW
BEGIN
    DECLARE v_abandoned_adoptions INT;
    DECLARE v_is_abandoned INT;

    SELECT COUNT(*) INTO v_is_abandoned
    FROM admissions i
    JOIN abandonments a ON a.admission_id = i.admission_id
    WHERE i.animal_id = NEW.animal_id;

    IF v_is_abandoned > 0 THEN
        SELECT COUNT(*) INTO v_abandoned_adoptions
        FROM adoptions ad
        JOIN admissions i ON i.animal_id = ad.animal_id
        JOIN abandonments a ON a.admission_id = i.admission_id
        WHERE ad.person_id = NEW.person_id
            AND YEAR(ad.adoption_datetime) = YEAR(NEW.adoption_datetime)
            AND MONTH(ad.adoption_datetime) = MONTH(NEW.adoption_datetime);

        IF v_abandoned_adoptions >= 2 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RN01: Una persona no puede adoptar más
de dos animales abandonados al mes';
        END IF;
    END IF;
END //
DELIMITER ;

```

Pruebas SQL

animales/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Tests negativos para AnimalesDB
--
USE AnimalesDB;

CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(p_id VARCHAR(20), p_message
VARCHAR(500), p_status VARCHAR(10))
BEGIN
    INSERT INTO test_results VALUES (p_id, SUBSTRING_INDEX(p_message, ':',
1), p_message, p_status, DEFAULT);
END //

CREATE OR REPLACE PROCEDURE p_test_uq01_species()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ01', 'UQ01: El
```

```

nombre de la especie debe ser único', 'PASS');
CALL p_populate();
INSERT INTO species (species_name) VALUES ('Canino');
CALL p_log_test('UQ01', 'ERROR: Se permitió duplicar una especie',
'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn02_abandonment()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN02A', 'RN02:
Un abandono requiere fecha, hora y lugar', 'PASS');
    CALL p_populate();
    INSERT INTO animals (animal_id, breed_id, chip) VALUES (8, 1, '88888');
    INSERT INTO admissions (admission_id, animal_id, delivery_datetime)
VALUES (8, 8, '2026-01-01 10:00:00');
    INSERT INTO abandonments (admission_id, abandonment_datetime, place)
VALUES (8, NULL, NULL);
    CALL p_log_test('RN02A', 'ERROR: Se permitió un abandono sin datos del
hallazgo', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn02_delivery()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN02E', 'RN02:
Una entrega debe identificar a la persona', 'PASS');
    CALL p_populate();
    INSERT INTO animals (animal_id, breed_id, chip) VALUES (8, 1, '88888');
    INSERT INTO admissions (admission_id, animal_id, delivery_datetime)
VALUES (8, 8, '2026-01-01 10:00:00');
    INSERT INTO deliveries (admission_id) VALUES (8);
    CALL p_log_test('RN02E', 'ERROR: Se permitió una entrega sin persona',
'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_adoptions()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01', 'RN01:
Máximo dos animales abandonados por persona y mes', 'PASS');
    CALL p_populate();
    INSERT INTO animals (animal_id, breed_id, chip) VALUES (8, 1, '88888');
    INSERT INTO admissions (admission_id, animal_id, delivery_datetime)
VALUES (8, 8, '2024-10-20 10:00:00');
    INSERT INTO abandonments (admission_id, abandonment_datetime, place)
VALUES (8, '2024-10-20 09:00:00', 'Parque Este');
    INSERT INTO adoptions (adoption_id, person_id, animal_id,
adoption_datetime) VALUES (8, 3, 8, '2024-10-25 10:00:00');
    CALL p_log_test('RN01', 'ERROR: Se permitió una tercera adopción de
animal abandonado', 'FAIL');
END //

```

```
CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        CALL p_log_test('POPULATE', 'ERROR: La carga inicial falló',
        'ERROR');
        SELECT * FROM test_results ORDER BY test_id;
    END;
    DELETE FROM test_results;
    CALL p_populate();
    CALL p_test_uq01_species();
    CALL p_test_rn02_abandonment();
    CALL p_test_rn02_delivery();
    CALL p_test_rn01_adoptions();
    SELECT * FROM test_results ORDER BY test_id;
    SELECT test_status, COUNT(*) total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;
CALL p_run_tests();
```

i Info

Versión PDF disponible